

Govor i scenario za odbranu

Dunja Janković 2020/0058

Razvoj interaktivnog asemblera i simulatora podskupa RISC-V instrukcija

Mentor: doc. dr Uroš Radenković

Plan: 10 slajdova, oko 9 minuta i 40 sekundi za govor i demonstraciju, uz oko 20 sekundi rezerve. Video od približno 1:45 već je uračunat u vreme slajda 7.

Ovo je predlog govora za vežbanje, ne tekst koji moraš da naučiš od reči do reči. Govori mirno, napravi pauzu posle važnih pojmova i pokaži ono o čemu pričaš. Vremena su okvirna: tačno trajanje potvrđuje se probnim izlaganjem sa videom.

U PPTX datoteci isti govor nalazi se u beleškama svakog slajda. Publika vidi samo kratke poruke i slike. Ne čitaj naglas podsetnike i uputstva za klikove.

Važno: video još nije ugrađen. Na slajdu 7 je stvarna slika završnog stanja aplikacije, koja može da posluži kao rezerva ili da se zameni snimljenim videom.

Vremenski raspored

Slajd	Tema	Vreme
1	Naslov i predstavljanje	0:00–0:20
2	Šta zapravo znači naslov rada?	0:20–1:40
3	Postojeći alati i cilj rada	1:40–2:25
4	Korišćene tehnologije	2:25–3:05
5	Šta aplikacija omogućava?	3:05–4:05
6	Kako su delovi povezani?	4:05–5:05
7	Primer rada aplikacije	5:05–7:05
8	Kako sam proverila da radi?	7:05–8:05
9	Šta je ostvareno i šta dalje?	8:05–9:25
10	Pitanja	9:25–9:40

Pitanja komisije dolaze posle izlaganja i nisu uključena u ovu minutažu.

Slajd 1 — Naslov i predstavljanje

Ciljno vreme: 0:20 | Ukupno: 0:00–0:20

Podsetnik: Predstavi temu, ne čitaj sve podatke sa naslovnog slajda.

Dobar dan svima. Ja sam Dunja Janković i predstaviću aplikaciju koju sam razvila u okviru diplomskog rada. Ona omogućava pisanje i praćenje izvršavanja jednostavnih RISC-V programa. Pre prikaza aplikacije, ukratko ću objasniti šta znači sam naslov rada.

Slajd 2 — Šta zapravo znači naslov rada?

Ciljno vreme: 1:20 | Ukupno: 0:20–1:40

Podsetnik: Prođi redom kroz pet pojmova. Napravi kratku pauzu pre završne rečenice.

Naslov zvuči stručno, ali je osnovna ideja jednostavna.

Procesor je deo računara koji izvršava naredbe — na primer, da sabere dva broja ili sačuva rezultat. Te naredbe zovemo instrukcije. RISC-V određuje koje instrukcije postoje i šta one rade, a u ovom radu podržan je odabrani deo njih — zato u naslovu piše „podskup”.

Asembler priprema napisane naredbe za izvršavanje, dok simulator oponaša rad procesora u programu. Dakle, nisam pravila fizički procesor, već aplikaciju koja prikazuje izvršavanje njegovih instrukcija.

Interaktivno znači da korisnik može da izvršava program korak po korak i posmatra šta se menja. Jednostavno rečeno, cilj je da ne vidimo samo konačan rezultat, već i kako je program do njega došao.

To je, na primer, zgodno kada želimo da razumemo kako program sabira brojeve: ne gledamo samo broj na kraju, nego i kako se zbir postepeno menja.

Slajd 3 — Postojeći alati i cilj rada

Ciljno vreme: 0:45 | Ukupno: 1:40–2:25

Podsetnik: Pokaži dva primera, pa pređi na cilj sopstvenog rešenja. Ne ulazi u spiskove mogućnosti.

Postoje i drugi alati koji omogućavaju ovakvo praćenje programa. U radu sam razmotrila RARS i Venus. RARS se koristi kao desktop aplikacija, dok je Venus dostupan kroz veb pregledač.

Izabrala sam ih zato što se i u njima mogu pisati i izvršavati mali RISC-V programi.

Kod njih sam posmatrala mogućnosti važne za moj zadatak: izvršavanje koracima, tačke prekida i pregled stanja. Cilj nije bio da napravim alat sa više funkcija od postojećih, već da samostalno realizujem celinu sa jasno određenim obimom. Zato sam se usredsredila na odabrane instrukcije i njihove rezultate.

Slajd 4 — Korišćene tehnologije

Ciljno vreme: 0:40 | Ukupno: 2:25–3:05

Podsetnik: Za svaku tehnologiju reci samo čemu služi i zašto je korisna.

Logiku aplikacije napisala sam u TypeScript-u. On mi pomaže da deo programerskih grešaka primetim još tokom pisanja koda. HTML i CSS koristila sam za strukturu, raspored i izgled ekrana.

Vite omogućava jednostavno pokretanje aplikacije tokom razvoja i pripremu za korišćenje. Vitest sam izabrala za automatske provere rezultata. Ukratko, jedan deo alata služi izradi aplikacije, a drugi proveriti da ona radi očekivano. Sama aplikacija se koristi u pregledaču.

Sve ove tehnologije služe izradi i proveriti aplikacije; korisnik pri radu ne mora da poznaje nijednu od njih.

Slajd 5 — Šta aplikacija omogućava?

Ciljno vreme: 1:00 | Ukupno: 3:05–4:05

Podsetnik: Pokaži levi panel, zatim desne panele i kontrole. Ne čitaj kod sa slike.

Ovako izgleda aplikacija. Levo se piše program, a desno se prate registri i memorija. Registre možemo da posmatramo kao mesta za vrednosti sa kojima procesor trenutno radi. U memoriji se vide podaci koje je program upisao.

Dugme *Step* izvršava jednu instrukciju. *Run* nastavlja izvršavanje, *Reset* vraća početno stanje, a tačkom prekida biramo mesto zaustavljanja. Tokom izvršavanja kod se ne uređuje, kako bi prikaz odgovarao programu koji upravo pratimo.

Ako rezultat nije očekivan, mogu da ponovim program i proveravam vrednosti posle svakog koraka.

Podržano je 15 instrukcija i 5 pseudo-instrukcija. Ove druge su kraći zapisi koji se prevode u podržane instrukcije. Za korisnika je najvažnije da može da napiše program, pokrene ga i vidi promene.

Slajd 6 — Kako su delovi povezani?

Ciljno vreme: 1:00 | Ukupno: 4:05–5:05

Podsetnik: Prati dijagram sleva nadesno. Objasni uloge, ne nazive funkcija ili datoteka.

Iza ovog ekrana nalaze se odvojeni delovi aplikacije. Korisnik prvo unese tekst programa. Parser pročita njegovu strukturu, a assembler proveriti instrukcije i pripremi ih za izvršavanje.

Možemo ih zamisliti kao nekoliko stanica kroz koje program prolazi, pri čemu svaka ima svoj zadatak.

Pripremljeni program zatim dobija simulator. On izvršava instrukcije i menja registre, memoriju i mesto sa kog se program nastavlja. Korisnički interfejs prikazuje dobijeno stanje. Ako se pojavi greška, korisnik dobija poruku umesto neispravnog rezultata.

Važna odluka bila je da logika i prikaz ostanu odvojeni. Tako sam mogla da proveravam izvršavanje bez pokretanja celog interfejsa i da menjam izgled bez menjanja pravila instrukcija. Sada ću taj tok pokazati na kratkom primeru.

Slajd 7 — Primer rada aplikacije

Ciljno vreme: 2:00 | Ukupno: 5:05–7:05

Podsetnik: Oko 10 sekundi uvoda, video 1:45 uz govor uživo, oko 5 sekundi za prelaz. Video još nije ugrađen; prikazana slika je rezerva.

Pre pokretanja videa:

Na kratkom videu pokazaću unos programa, pojedinačne korake, zaustavljanje i konačan rezultat. Primer je sabiranje brojeva od 1 do 5.

0:00–0:10 od početka videa:

Za primer sam izabrala sabiranje brojeva od 1 do 5. Očekivani rezultat je 15, pa možemo lako da pratimo da li program radi ono što želimo.

0:10–0:18 od početka videa:

Najpre proveravam zapis programa i pripremam instrukcije za izvršavanje. Pojavljuje se oznaka sledeće instrukcije.

0:18–0:42 od početka videa:

Svakim pritiskom na *Step* izvršava se jedna instrukcija. Prvo se postavljaju početne vrednosti, a zatim se obavlja prvo sabiranje. Desno vidimo da je tekući zbir postao 1.

0:42–1:02 od početka videa:

Sada postavljam tačku prekida. Time unapred biram mesto na kome program treba da se zaustavi. Ne moram ručno da prolazim kroz svaki korak: mogu da pokrenem program i pogledam stanje kada dođe do izabranog mesta.

1:02–1:16 od početka videa:

Ponovnim pokretanjem nastavljam od mesta gde je program stao. Zbir je sada 3, a program se ponovo zaustavio na istoj tački u petlji.

1:16–1:31 od početka videa:

Uklanjam tačku prekida i dopuštam programu da završi. Dobijeni zbir je 15, a isti rezultat je upisan i u memoriju.

1:31–1:45 od početka videa:

Program mogu da vratim na početak i ponovo pokrenem. Dobijam isti rezultat, a završni prikaz ostaje dostupan za pregled.

Posle videa:

Tako se vidi ceo put od napisanog programa do rezultata.

Ako video nije dostupan:

Ako video nije dostupan: Na slici je završno stanje primera. Program sabira brojeve od 1 do 5, pa očekujemo rezultat 15. Levo se vidi program, a desno njegove završne vrednosti.

Tokom izvršavanja mogla sam da prolazim instrukciju po instrukciju pomoću *Step-a*, ili da postavim tačku prekida i prepustim programu da sam dođe do nje. Tačka u petlji omogućava da se stanje pogleda posle svakog prolaza. Kada uklonim tačku prekida, *Run* dovršava program.

Na kraju je zbir 15 u registru i memoriji. *Reset* omogućava ponavljanje od početka. I bez videa, ova slika prikazuje konačan rezultat, dok se način upravljanja vidi na kontrolama ispod programa.

Slajd 8 — Kako sam proverila da radi?

Ciljno vreme: 1:00 | Ukupno: 7:05–8:05

Podsetnik: Naglasak je na proverljivosti. Broj 113 odnosi se na testove logike, ne na broj UI proveru.

Nije dovoljno da aplikacija samo izgleda kao da radi. U projektu postoji 113 automatskih testova za parser, assembler i model procesora, i pri završnoj proveru svi su prošli.

To su provere koje porede dobijeni rezultat sa očekivanim. Pored ispravnih programa, proveravala sam granične vrednosti, neispravne instrukcije i nedozvoljene pristupe memoriji. Tako se traže i greške koje ne bismo primetili samo pokretanjem jednog primera.

Dodatno je proveren rad korisničkih kontrola i prikaza. To ne znači da je ispitan svaki mogući slučaj, ali postoje ponovljive provere najvažnijih pravila, na koje se mogu osloniti i pri daljim izmenama aplikacije.

Kada kasnije izmenim aplikaciju, iste testove mogu ponovo da pokrenem i proverim da nisam pokvarila ranije ponašanje.

Slajd 9 — Šta je ostvareno i šta dalje?

Ciljno vreme: 1:20 | Ukupno: 8:05–9:25

Podsetnik: Završi pozitivno, ali jasno odvoji postojeće mogućnosti od budućih ideja.

Najvažniji rezultat je celina u kojoj korisnik može da napiše program, izvrši ga i prati šta se dešava. Ostvarene su dogovorene mogućnosti: pojedinačni koraci, rad do tačke prekida, izvršavanje do kraja i prikaz registara i memorije.

Obim je namerno ograničen. Ne simuliram svaki detalj pravog procesora, već ponašanje odabranih RISC-V instrukcija. To je omogućilo da se ceo postupak izrade i provere objasni jasno.

Dalji razvoj mogao bi da donese više instrukcija i složenije primere. Korisni dodaci bili bi i učitavanje i čuvanje programa, kao i vraćanje jednog koraka, kada želimo ponovo da pogledamo prethodno stanje.

Takve dopune bih uvodila postepeno, uz nove provere, kako bi rešenje ostalo pregledno. Važno mi je da korisnik uvek može da poveže naredbu koju je napisao sa promenom koju vidi na ekranu.

Aplikacija je namenjena demonstraciji i učenju, a njenu korist za korisnike trebalo bi dodatno ispitati u praksi. Za ovaj rad dobila sam zaokruženo rešenje i osnovu koju mogu dalje da proširujem.

Slajd 10 — Pitanja

Ciljno vreme: 0:15 | Ukupno: 9:25–9:40

Podsetnik: Zastani, pogledaj publiku i sačekaj pitanja. Pitanja nisu deo planiranih deset minuta izlaganja.

Time bih završila izlaganje. Hvala vam na pažnji. Spremna sam za vaša pitanja.

Snimanje videa: tačan redosled

Pokreni aplikaciju pre snimanja. Snimaj samo njen prozor, bez terminala, instaliranja i drugih programa. Isključi obaveštenja. Video treba da bude bez muzike i bez unapred snimljenog govora, jer ga komentarišeš uživo.

Uvećaj prikaz u pregledaču tako da su kod i vrednosti čitljivi, ali da nijedan panel ili dugme nije odsečeno. Na računaru sa ekranom 1920×1080 probaj uvećanje 125–150%, pa prilagodi ako raspored ne staje. Snimaj u MP4 formatu, po mogućstvu sa H.264 kodiranjem.

0:00–0:10 — Izaberi primer „Zbir brojeva 1-5”.

0:10–0:18 — Klikni Asembliraj i kratko zadrži prikaz.

0:18–0:42 — Četiri puta polako klikni *Step*. Posle četvrtog koraka $x_1=1$, $x_2=1$, $x_3=6$; marker je na liniji 8.

0:42–1:02 — Klikni na liniju 7 sa *ADD*, zatim *Run*. Očekuj $x_1=1$, $x_2=2$, $x_3=6$ i marker na liniji 7.

1:02–1:16 — Ponovo klikni *Run*. Očekuj $x_1=3$, $x_2=3$, $x_3=6$ i ponovno zaustavljanje na liniji 7.

1:16–1:31 — Dok je marker na liniji 7, klikni *Breakpoint* da ukloniš tačku, pa *Run*.

1:31–1:45 — Klikni *Reset*, zatim *Run*. Poslednjih nekoliko sekundi zadrži konačno stanje: $x_1=15$, $x_2=6$, $x_3=6$, $\text{memorija}[0]=15$.

Za *breakpoint* klikni baš na liniju 7 sa *ADD*, a ne na dugme *Breakpoint* dok marker stoji na liniji 8. Dugme kasnije koristiš za uklanjanje tačke kada se program već zaustavio na liniji 7.

Poslednjih nekoliko sekundi zadrži rezultat 15. Ne ubrzavaj video da bi stao u zadato vreme: važnije je da publika primeti promene. Ako je predugačak, ukloni prazne pauze na početku i kraju ili ponovi snimanje sa navedenim redosledom.

Umetanje videa i rezervna varijanta

Sačuvaj kopiju prezentacije pre izmene. Na slajdu 7 izaberi samo sliku aplikacije (u oknu za izbor označena je kao rezervna slika za zamenu videom), a ne pozadinu šablona.

U PowerPoint-u izaberi Umetanje → Video → Ovaj uređaj i ubaci MP4 datoteku. Video ugradi u prezentaciju, umesto da ostane samo veza ka datoteci na drugom računaru. Postavi ga umesto slike i podesi pokretanje na klik.

Za bolju čitljivost možeš da uključiš reprodukciju preko celog ekrana. Proveri da se po završetku vraćaš na slajd, pa prelaziš na testiranje. Obavezno isprobaj celu prezentaciju sa videom na računaru koji ćeš koristiti na odbrani.

Sačuvaj i verziju sa rezervnom slikom. PDF prezentacije je dodatna statička rezerva: u njemu se video ne reprodukuje. Ako video ne radi, koristi alternativni govor sa slajda 7 i jasno reci da pokazuješ sliku završnog stanja.

Podsetnik za poslednje vežbanje

1. [0:00] Predstavi se i najavi da ćeš prvo objasniti naslov.
2. [0:20] Naredbe → RISC-V → assembler → simulator → praćenje koraka.
3. [1:40] RARS i Venus; cilj je sopstveno rešenje ograničenog obima.
4. [2:25] TypeScript — logika; HTML/CSS — izgled; Vite — razvoj; Vitest — proveru.
5. [3:05] Pokaži program levo, registre i memoriju desno i kontrole ispod.
6. [4:05] Tekst → priprema → izvršavanje → prikaz. Logika i interfejs su odvojeni.
7. [5:05] Video: četiri koraka → *breakpoint* → nastavak → zbir 15 → *Reset* i novo pokretanje.
8. [7:05] 113 uspešnih testova logike, uz dodatnu proveru interfejsa.
9. [8:05] Ostvareni ciljevi, jasan obim i realni predlozi unapređenja.
10. [9:25] Zahvali se, zastani i sačekaj pitanja.

Vežbaj najmanje jednom uz merenje vremena, zajedno sa videom. Ako govoriš prebrzo, dodaj kratke pauze posle definicija i pokazanih promena. Ako prelaziš desetak minuta, skрати objašnjenja postojećih alata i tehnologija, a ne najvažnije delove demonstracije.